

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 02 -

REGRESSÃO LINEAR E REGULARIZAÇÃO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	10
MERCADO, CASES E TENDÊNCIAS	15
O QUE VOCÊ VIU NESTA AULA?	18
REFERÊNCIAS.....	20

LEANDRO

O QUE VEM POR AÍ?

Veremos como manter modelos lineares em produção confiáveis por meio de práticas de observabilidade inspiradas em sistemas distribuídos. Veremos três checagens centrais: vitalidade, prontidão e inicialização, além de um controle explícito da “capacidade estatística” via regularização. A ideia é detectar rapidamente quando um modelo degrada do ponto de vista do negócio e agir de forma objetiva, evitando dependência de intuição ou intervenções tardias.

A vitalidade avalia se o modelo permanece estatisticamente saudável, monitorando métricas (RMSE, MAE, R^2), resíduos e diagnósticos como heterocedasticidade, normalidade, multicolinearidade e influência. Quando falha, correções automatizadas entram em ação, como re-treino com Ridge, Lasso ou Elastic Net, ou rollback para um baseline mais simples. Já a prontidão decide se o modelo pode receber tráfego, garantindo alinhamento entre pipeline de pré-processamento, esquema de features, versões de artefatos e resultados de shadow testing, prevenindo exposição prematura em produção.

A inicialização reconhece que alguns modelos precisam de tempo para “aquecer” pipelines complexos, evitando falsos positivos de falha. Por fim, trataremos da regularização como mecanismo de alocação de recursos estatísticos: sem penalização há overfitting; com excesso, viés. Ridge, Lasso e Elastic Net oferecem trade-offs distintos, calibrados por validação cruzada e critérios informativos. Governança contínua, checagens automáticas e regularização bem dosada tornam modelos lineares simples mais robustos, previsíveis e adequados a decisões em produção.

HANDS ON

Vamos experimentar os conceitos de regressão linear e regularização de forma simples, usando Python com scikit-learn e statsmodels para montar um pipeline direto e repetível. Escolhemos Python porque tem um ecossistema rico para dados e facilita equilibrar desempenho e interpretação sem complicação. Com ele, geramos um conjunto sintético com variáveis numéricas e categóricas, treinamos uma regressão linear etc.

Suponha que temos um conjunto de dados tabular com variáveis numéricas e categóricas e queremos construir um serviço de previsão baseado em regressão. Cada coluna do dataset tem um papel claro: as numéricas (renda, quartos, latitude, longitude) trazem magnitude e escala próprias; as categóricas (bairro, condicao) representam grupos e estados discretos. Nosso objetivo é preparar essas entradas, treinar um modelo de regressão linear como baseline e, quando necessário, aplicar regularização (Ridge, Lasso ou Elastic Net) para tornar as estimativas mais estáveis e reduzir o risco de overfitting.

```
# Setup
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, KFold,
GridSearchCV, TimeSeriesSplit
from sklearn.preprocessing import StandardScaler,
OneHotEncoder, PolynomialFeatures
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error,
mean_absolute_error, r2_score
from sklearn.linear_model import LinearRegression, Ridge,
Lasso, ElasticNet
import statsmodels.api as sm

np.random.seed(42)
N = 3000
X_num = pd.DataFrame({
    'renda': np.random.normal(50000, 10000, N),
    'quartos': np.random.randint(1, 6, N),
    'latitude': np.random.uniform(32, 42, N),
    'longitude': np.random.uniform(-124, -114, N)
})
X_cat = pd.DataFrame({
    'bairro': np.random.choice(['A', 'B', 'C', 'D'], size=N),
```

```

        'condicao': np.random.choice(['nova', 'usada'], size=N)
    })
    X = pd.concat([X_num, X_cat], axis=1)
    y = 100000 + 3*X_num['renda'] + 10000*X_num['quartos'] -
        5000*(X_num['latitude']-37)**2 + np.random.normal(0, 50000, N)

    X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=0.2, random_state=42)

```

Código-fonte 1 -# Setup import numpy as np
 Fonte: Elaborado pelo autor (2026)

No snippet a seguir, separamos as colunas numéricas (renda, quartos, latitude, longitude) das categóricas (bairro, condicao) e montamos um pré-processamento com ColumnTransformer: nas numéricas aplicamos StandardScaler (para colocar tudo na mesma escala) e, nas categóricas, OneHotEncoder com handle_unknown='ignore' (para que categorias novas não quebrem o fluxo). Em seguida, criamos um pipeline que junta esse pré-processamento ao modelo LinearRegression, garantindo que tudo rode na mesma ordem e sem “atalhos” que possam vaziar informação do teste para o treino. Depois treinamos com `ols.fit(X_train, y_train)` e geramos previsões para treino e teste.

Por fim, imprimimos um resumo simples com RMSE_train, RMSE_test, MAE_test e R2_test: o RMSE mostra o erro médio na unidade do alvo, o MAE é uma leitura direta do erro médio (menos sensível a extremos) e o R^2 indica quanto da variação o modelo explica. A interpretação prática é: se o RMSE de teste estiver perto do de treino e o R^2 de teste for bom, o baseline está saudável; se houver diferença grande, é sinal para ajustar os dados ou testar versões com regularização (Ridge, Lasso, Elastic Net).

```

    num_cols = ['renda', 'quartos', 'latitude', 'longitude']
    cat_cols = ['bairro', 'condicao']

    preprocess = ColumnTransformer([
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
    ])

    ols = Pipeline([
        ('prep', preprocess),
        ('model', LinearRegression())
    ])

```

```

ols.fit(X_train, y_train)

pred_tr = ols.predict(X_train)
pred_te = ols.predict(X_test)
print({
    'RMSE_train': mean_squared_error(y_train, pred_tr,
squared=False),
    'RMSE_test': mean_squared_error(y_test, pred_te,
squared=False),
    'MAE_test': mean_absolute_error(y_test, pred_te),
    'R2_test': r2_score(y_test, pred_te)
})

```

Código-fonte 2 - num_cols = ['renda','quartos','latitude','longitude'] // cat_cols = ['bairro','condicao']
(Python)

Fonte: Elaborado pelo autor (2026)

No snippet a seguir, criamos três pipelines que reaproveitam o mesmo pré-processamento (StandardScaler para numéricas e OneHotEncoder para categóricas) e trocam apenas o modelo: Ridge, Lasso e ElasticNet. Em seguida, definimos uma validação cruzada simples (KFold com 5 dobras, embaralhando os dados e random_state=42 para repetibilidade) e três grades de busca de hiperparâmetros: alpha para Ridge, alpha para Lasso e, no ElasticNet, tanto alpha quanto l1_ratio (que controla o equilíbrio entre L1 e L2). Cada GridSearchCV avalia combinações usando a métrica de RMSE (via scoring='neg_root_mean_squared_error', por isso o resultado é negativo e o código imprime o valor positivo com o sinal invertido).

Depois de ajustar cada grade com fit(X_train, y_train), imprimimos os melhores hiperparâmetros e o RMSE médio de validação cruzada para cada método. A leitura prática é direta: quanto menor o RMSE, melhor a combinação de penalização para aquele modelo; compare os três números para decidir se vale manter o baseline ou usar uma regularização (Ridge para estabilizar coeficientes em alta correlação, Lasso para seleção de variáveis, ElasticNet para cenários com grupos de preditores correlacionados).

```

ridge = Pipeline([('prep', preprocess), ('model',
Ridge())])
lasso = Pipeline([('prep', preprocess), ('model',
Lasso(max_iter=10000))])
elastic = Pipeline([('prep', preprocess), ('model',
ElasticNet(max_iter=10000))])

```

```

cv      =      KFold(n_splits=5,      shuffle=True,      random_state=42)
pr      =      {'model__alpha':[0.1,1,10,100]}
pl      =      {'model__alpha':[0.001,0.01,0.1,1]}
pe      =      {'model__alpha':[0.001,0.01,0.1,1],
'model__l1_ratio':[0.2,0.5,0.8]}

GR      =      GridSearchCV(ridge,      pr,      cv=cv,
scoring='neg_root_mean_squared_error')
GL      =      GridSearchCV(lasso,      pl,      cv=cv,
scoring='neg_root_mean_squared_error')
GE      =      GridSearchCV(elastic,      pe,      cv=cv,
scoring='neg_root_mean_squared_error')

for      g      in      [GR,      GL,      GE]:
    g.fit(X_train,      y_train)

print('Ridge:',      GR.best_params_,      -GR.best_score_)
print('Lasso:',      GL.best_params_,      -GL.best_score_)
print('Elastic:',      GE.best_params_,      -GE.best_score_)

```

Código-fonte 3 - ridge = Pipeline([('prep', preprocess), ('model', Ridge())]) (Python)
Fonte: Elaborado pelo autor (2026)

O trecho a seguir cria um pré-processamento que amplia a capacidade do modelo com termos polinomiais de grau 3 nas colunas numéricas, mantendo as categóricas com one-hot. Primeiro, as variáveis numéricas passam por StandardScaler para ficarem na mesma escala e, em seguida, ganham interações e potências com PolynomialFeatures(degree=3, include_bias=False), evitando duplicar o intercepto. As colunas categóricas são codificadas com OneHotEncoder(handle_unknown='ignore'), o que impede falhas quando surgem categorias novas. Tudo isso é encadeado em um Pipeline com LinearRegression, garantindo que as etapas ocorram sempre na mesma ordem e que o treino não “vaze” informações para o teste. Após ajustar com fit e prever em X_test, o código imprime o RMSE de teste como medida direta de erro.

Ao aumentar a complexidade com polinômios, o risco de overfitting sobe: o modelo passa a se ajustar demais aos detalhes do treino e pode perder desempenho fora da amostra. Por isso, é importante comparar RMSE do treino e do teste, usar validação cruzada e, se necessário, aplicar regularização para controlar a variância. A combinação mais comum é manter o mesmo pré-processamento e trocar o estimador para Ridge, Lasso ou Elastic Net, ajustando hiperparâmetros com GridSearchCV.

Ridge ajuda a estabilizar coeficientes quando há muitas interações correlacionadas; Lasso pode reduzir o número de termos, zerando os menos relevantes; Elastic Net equilibra ambos em cenários com grupos de features correlacionadas. Em resumo, polinômios podem melhorar a previsão, mas só com controle de complexidade e avaliação honesta o ganho se mantém no mundo real.

```
poly_prep = ColumnTransformer([
    ('num', Pipeline([('scaler', StandardScaler()), ('poly',
    PolynomialFeatures(degree=3, include_bias=False))]),
    num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
])
poly_ols = Pipeline([('prep', poly_prep), ('model',
LinearRegression())])
poly_ols.fit(X_train, y_train)
print('RMSE_test (poly grau 3):', mean_squared_error(y_test,
poly_ols.predict(X_test), squared=False))
```

Código-fonte 4 - Pré-processamento que amplia a capacidade do modelo com termos polinomiais de grau 3 nas colunas numéricas (Python)

Fonte: Elaborado pelo autor (2026)

No trecho a seguir, criamos um “caminho de regularização” para o Lasso: geramos 30 valores de alpha em escala logarítmica (de 10^{-3} a 1), montamos um pipeline com o mesmo pré-processamento (padronização nas numéricas e one-hot nas categóricas) e trocamos apenas o estimador para Lasso(alpha=a). Para cada alpha, ajustamos o modelo e guardamos o vetor de coeficientes. No fim, empilhamos tudo em um array (coefs) e imprimimos sua forma. Essa forma é (n_alphas, n_features_transformadas): cada linha mostra como os coeficientes se comportam para um nível de penalização e cada coluna corresponde a uma feature depois do ColumnTransformer. À medida que alpha cresce, a penalização L1 fica mais forte, muitos coeficientes encolhem e alguns vão a zero, revelando o padrão de sparsidade e como a complexidade do modelo diminui.

```
alphas = np.logspace(-3, 0, 30)
coefs = []
for a in alphas:
    model = Pipeline([('prep', preprocess), ('model',
    Lasso(alpha=a, max_iter=10000))])
    model.fit(X_train, y_train)
    coefs.append(model.named_steps['model'].coef_)
coefs = np.array(coefs)
```



```
print('Shape dos coeficientes ao longo do caminho:',  
      coefs.shape)
```

Código-fonte 5 – “Caminho de regularização” para o Lasso (Python)

Fonte: Elaborado pelo autor (2026)

Usamos um pipeline reproduzível com pré-processamento via ColumnTransformer (StandardScaler nas numéricas e OneHotEncoder nas categóricas) e um baseline OLS para medir desempenho com RMSE, MAE e R^2 ; o diagnóstico com statsmodels (sumário e resíduos) ajuda a detectar multicolinearidade e padrões de erro. Quando há risco de overfitting ou instabilidade, aplicamos regularização (Ridge, Lasso, Elastic Net) e calibramos hiperparâmetros com KFold/GridSearchCV, observando também o caminho de coeficientes para entender a sparsidade. Com essas etapas, comparamos resultados em teste e escolhemos o melhor equilíbrio entre simplicidade, estabilidade e generalização.

SAIBA MAIS

A seguir, mergulharemos nos fundamentos e nas sutilezas da regressão linear e das técnicas de regularização, conectando matemática, diagnóstico estatístico e engenharia de ML a cenários reais de produção. A proposta é manter a conversa leve, mas precisa e densa em conteúdo: vamos discutir não apenas como treinar, mas por que determinadas escolhas funcionam (ou falham) quando o modelo encontra dados ruidosos, restrições de latência/custo e requisitos de confiabilidade no mundo real.

Veremos como práticas de pré-processamento (normalização/padronização, tratamento de outliers e colinearidade), métricas (RMSE, MAE, R^2) com leitura operacional, diagnóstico com statsmodels (análise de resíduos, heterocedasticidade, multicolinearidade, testes de hipótese) e penalizações Ridge/Lasso/Elastic Net se combinam para construir e operar regressões em escala, com estabilidade e boa generalização.

Amarraremos esses conceitos a rotinas de MLOps: validação cruzada e tuning de hiperparâmetros, pipelines reproduzíveis, monitoramento de desempenho em produção (drift, degradação de métricas) e estratégias de deployment (canary/shadow/A-B) que mantêm o modelo robusto sob variação de carga e mudança de dados. A ideia é reforçar os princípios teóricos que sustentam essas ferramentas e mostrar como otimizar seu uso para operar regressões com confiança em ambientes reais

Pré-processamento e pipeline: preparando o terreno para generalizar

O pré-processamento organiza o caminho dos dados brutos até a previsão, separando colunas numéricas (ex.: renda, quartos, latitude, longitude) e categóricas (ex.: bairro, condicao) e aplicando transformações adequadas: StandardScaler nas numéricas para padronizar escala e OneHotEncoder nas categóricas para representar categorias como variáveis binárias, tolerando valores novos com `handle_unknown='ignore'`. Encadear tudo em Pipeline e ColumnTransformer fixa a ordem das etapas e reduz o risco de “atalhos” que, em produção, levam a erros de esquema ou a resultados inconsistentes.

Além da limpeza e padronização, o pipeline atua como contrato de interface: determina como o modelo deve “ver” os dados e impede que transformações aprendam com o conjunto de teste (leakage). Isso garante reprodutibilidade entre ambientes (notebook, API, lotes) e simplifica o versionamento, já que as etapas e seus parâmetros ficam registradas de ponta a ponta. Em serviços long-running, esse desenho facilita checagens de inicialização (carregar artefatos, validar esquema) e de prontidão (rodar smoke tests), antes do modelo começar a atender tráfego.

Métricas e seus efeitos: RMSE, MAE e R^2

RMSE, MAE e R^2 são os observadores da saúde do modelo. O RMSE (raiz do erro quadrático médio) está na mesma unidade do alvo e penaliza erros grandes; o MAE (erro absoluto médio) é mais robusto a outliers e dá uma leitura direta do erro típico; o R^2 quantifica a fração da variância do alvo explicada pelas features. Avaliar essas métricas em treino e teste permite identificar overfitting (erro de teste maior e R^2 menor) e underfitting (erros altos em ambos).

Na prática, evitamos decisões baseadas em um único corte treino/teste, preferindo validação cruzada (k-fold) para obter médias e variâncias das métricas fora da amostra. Em produção, metas operacionais (faixas de RMSE/MAE e mínimos de R^2) se tornam limites de prontidão e vitalidade: quando as métricas saem da banda de tolerância de forma sustentada, disparamos correções (retreino, rollback, ajuste de hiperparâmetros) com critérios claros e repetíveis.

Diagnóstico com statsmodels: OLS, resíduos e testes

Mesmo com boas métricas, olhar “por dentro” evita surpresas. Um ajuste OLS com statsmodels fornece sumários de coeficientes, erros padrão e significância, além de facilitar a inspeção de resíduos. Testes como Breusch–Pagan (heterocedasticidade) e Shapiro–Wilk (normalidade) e indicadores como VIF, leverage e distância de Cook revelam multicolinearidade, padrões em resíduos e pontos influentes que podem dominar o ajuste — sinais de que o modelo precisa de correções.

Essas análises orientam ações concretas: aplicar transformações (ex.: log), tratar outliers, revisar engenharia de variáveis ou adotar regularização para reduzir variância e estabilizar estimativas. O objetivo é garantir que o modelo não apenas “cabe” nos dados, mas que suas suposições básicas estejam razoavelmente satisfeitas, preservando a confiabilidade da previsão quando o serviço enfrentar novas entradas.

Regularização: o controle da complexidade

Regularização funciona como limites que impedem o modelo de consumir toda a capacidade de generalização. Ridge (L2) adiciona $\lambda \|\beta\|_2^2$, encolhe coeficientes e melhora o condicionamento quando $X^T X$ é mal-condicionado; raramente zera termos, sendo útil em alta correlação. Lasso (L1) adiciona $\lambda \|\beta\|_1$, promovendo sparsidade — muitos coeficientes vão a zero — o que atua como seleção de variáveis. Elastic Net combina L1/L2 via α , equilibrando seleção e estabilidade em grupos de preditores correlacionados.

Aplicar penalização exige parcimônia: pouca penalização aumenta variância (overfitting); penalização demais eleva viés (perda de sinal). Por isso, padronizar números antes de Lasso/Elastic Net é obrigatório e a escolha de λ e α deve vir de validação cruzada. Em pipelines com muitas dummies e interações, a regularização reduz oscilações e melhora a robustez, sem depender apenas de remoções manuais de variáveis.

Seleção de hiperparâmetros e validação

Para escolher λ e α , usamos protocolos de validação como KFold e GridSearchCV (ou estratégias de busca aleatória/baía). No scikit-learn, a métrica de RMSE aparece como `neg_root_mean_squared_error` (negativa por convenção) e o valor reportado é invertido na leitura. O objetivo é comparar combinações de hiperparâmetros e selecionar aquela com menor RMSE médio de validação, garantindo que o ganho não seja fruto de acaso.

Em problemas com temporalidade, substituímos KFold por TimeSeriesSplit para respeitar a ordem dos dados e evitar “vazamento do futuro”. Para avaliações

mais rigorosas, a validação aninhada (nested CV) separa tuning e aferição, fornecendo estimativas de desempenho menos otimistas. Uma vez definidos os hiperparâmetros, o modelo final é treinado no conjunto completo de treino e promovido conforme critérios de prontidão.

Adicionar `PolynomialFeatures` amplia a flexibilidade para capturar não linearidades, mas eleva o risco de overfitting, principalmente quando combinado com `OneHotEncoder`, que aumenta o número de colunas. A regra prática é comparar RMSE/MAE em treino e teste (e CV) e, se necessário, aplicar Ridge, Lasso ou Elastic Net para “segurar” a variância e manter apenas termos que agregam sinal.

Quando o ganho com polinômios é real, ele precisa ser sustentável fora da amostra. Por isso, além das métricas, vale inspecionar o comportamento dos coeficientes e utilizar caminhos de regularização (ex.: varrer alphas no Lasso) para ver onde a sparsidade estabiliza. Se o benefício não se mantém, é melhor voltar ao baseline linear e investir em engenharia de variáveis mais informativa e simples.

Operação em produção: prontidão, vitalidade e inicialização do pipeline

Antes de liberar o modelo ao tráfego, rodamos um check de prontidão: o pipeline carrega na ordem correta, o esquema de features coincide com o treinado, versões de artefatos batem e o shadow testing confirma que RMSE/MAE/R² estão dentro das faixas acordadas frente à versão atual. Prontidão não “conserta” lógica de negócio; ela apenas confirma que o serviço pode participar do roteamento de previsões sem quebrar.

Em operação contínua, vitalidade monitora métricas e resíduos para detectar degradações sustentadas (não picos momentâneos). Quando limites são excedidos por tempo suficiente, acionamos re-treino, rollback ou ajuste de penalização. Para stacks que demoram a aquecer (muitas dummies, carregamento de transformadores e coeficientes), uma janela de inicialização evita falsos positivos: o serviço valida artefatos e roda verificações rápidas antes de aceitar tráfego.

Padronize variáveis numéricas antes de Lasso/Elastic Net, mantenha todas as transformações dentro do Pipeline para evitar leakage e versione tanto dados quanto artefatos (scalers, encoders, coeficientes). Documente o esquema de features,

monitore drift de distribuição e configure alertas para quedas em RMSE/MAE/ R^2 e sinais de condicionamento ruim (ex.: VIF alto, resíduos com padrão).

Diferencie ruído temporário de mudança estrutural antes de retreinar ou apertar λ/α . Inspeccione pontos influentes (Cook, leverage) e revise a engenharia de variáveis quando padrões persistem. Por fim, mantenha um baseline confiável para fallback e um processo claro de promoção/rollback, garantindo previsibilidade e auditabilidade no ciclo de vida do modelo.

EMANIP

MERCADO, CASES E TENDÊNCIAS

Para fechar a discussão da Aula de Regressão Linear e Regularização, vamos olhar para exemplos do mundo real em que esses conceitos — OLS, Ridge/L2, Lasso/L1, Elastic Net e SGD — foram cruciais, seja como histórias de sucesso, seja como aprendizados por falhas. A ideia é ligar teoria e prática: reduzir overfitting via regularização, lidar com multicolinearidade, evitar data leakage e escalar treino/inferência com escolhas de algoritmo e plataforma.

Em 2025, um time de ciência de dados auditou seus pipelines e descobriu data leakage em modelos de regressão para risco financeiro: normalizações e seleção de variáveis eram feitas antes do split treino/teste e havia features com informação futura (target leakage) entrando no treinamento. Isso inflava métricas offline e derrubava performance em produção. A correção adotou três frentes: (1) isolar features potencialmente sensíveis em pipelines dedicados (fit do scaler e do encoder apenas no conjunto de treino; aplicação no teste), (2) bloquear uso de variáveis derivadas do alvo e (3) inspeções automáticas com checklists e ferramentas de detecção de leakage. Resultado: métricas mais conservadoras offline, porém generalização estável em produção.

Prática recomendada que emergiu dessa auditoria: tratar data leakage como risco operacional — padronizar splits temporais quando houver autocorrelação, versionar transformações e separar governança de dados (acesso, logs, auditoria) da governança de modelagem (ordem das etapas, validação cruzada, travas de features). Guias e pesquisas recentes reforçam que leakage ocorre por pré-processamento fora de ordem, contaminação de treino/teste e uso de “sinais futuros”, devendo ser prevenido estruturalmente, não apenas em “patches” de código.

Um case clássico de redução de custo operacional veio de um e-commerce que rodava regressões com centenas de preditores tabulares. Ao migrar de OLS para Lasso e, depois, para Elastic Net, o time reduziu a dimensionalidade efetiva (coeficientes zerados via L1), deixou o scoring mais leve e melhorou a interpretabilidade — sem perda de acurácia no teste. O raciocínio: L1 faz seleção automática de variáveis; L2 estabiliza sob multicolinearidade; Elastic Net balanceia

ambos (controlado por α e $l1_ratio$). Em plataformas como scikit-learn, isso cai na prática com Lasso(L1), Ridge(L2) e ElasticNet(L1+L2), calibrados via cross-validation (e sempre com padronização prévia de features).

Por que isso economiza? Menos features $\rightarrow$ menos I/O, menos cache churn e menos custo de manutenção (ETLs e catálogos ficam mais simples). Técnicas de regularização reduzem variância e evitam overfitting ao custo de um pouco mais de viés, exatamente o movimento esperado no bias-variance trade-off (ganho de generalização). Em cenários com correlações fortes, Elastic Net costuma outperformar Lasso puro (evita “cortar demais”) e Ridge puro (evita “manter demais”), sendo a “rede de segurança” quando há muitos preditores correlacionados.

Uma startup de varejo viu seus coeficientes de uma regressão de preços explodirem e inverterem sinal após um update de features; o problema era a multicolinearidade (variáveis altamente correlacionadas entre si). O efeito: coeficientes instáveis, erros-padrão inflados e conclusões sem robustez. A recuperação veio em três passos: (1) diagnóstico com VIF (Variance Inflation Factor), (2) remoção/combinção de preditores redundantes (ou PCA, quando cabível) e (3) adoção de Ridge como baseline regularizado para estabilizar estimativas.

Ferramentas e guias práticos concordam: VIF alto sinaliza que o coeficiente “herda” variância dos vizinhos; correções incluem dropar uma das variáveis, agregar (PCA/fator) ou passar a L2 (Ridge) em produção. Em pipelines modernos é útil automatizar alertas de VIF e retreinar com penalização quando o VIF médio superar um limiar (ex.: >5 ou >10 , conforme política).

Para apps de áudio/vídeo e experiências em tempo real, equipes relatam ganhos ao rodar inferência on-device com Core ML — eliminando latência de rede e mantendo privacidade. Em regressões simples (e modelos lineares em geral), o on-device aproveita CPU/GPU/Neural Engine e integra com Vision/Natural Language/Sound Analysis, entregando respostas de baixa latência (sub-100ms em hardware recente), e previsões estáveis mesmo offline. O Core ML fornece ferramentas para otimizar e perfilar modelos, além de personalização on-device quando necessário.

Em termos de QoS, isso equivale a “nós exclusivos” para workloads críticos no cluster de dispositivos do usuário: inferência local, pipeline padronizado e uso eficiente do silício. A documentação da Apple e guias técnicos destacam o ganho de responsividade e privacidade — um paralelo direto ao isolamento de cargas em clusters, só que no ecossistema iOS/macOS.

Em bases pequenas, a solução fechada da OLS é elegante: $\hat{\beta} = (X^T X)^{-1} X^T y$. Em bases grandes (milhões de linhas, milhares de features), inverter $X^T X$ fica caro (tempo e memória) e o caminho prático é usar solvers iterativos como Gradient Descent/SGD (com mini-batch, regularização e escala de features). Isso troca “exatidão única” por convergência rápida e escalável, especialmente em dados esparsos e de alto volume.

Na prática: Ridge com solver='sag'/'saga' (ou lsqr/sparse_cg) escala bem; SGDRegressor com perda quadrática e penalização L2 resolve o mesmo problema via outro caminho, adequado para 10^5+ exemplos/features. Materiais acadêmicos recentes mostram análises simplificadas de SGD com weight averaging e decompõem risco em viés+variância, reforçando porque métodos iterativos são o cavalo de batalha na indústria.

O QUE VOCÊ VIU NESTA AULA?

Esta aula apresentou um guia completo de regressão linear e mostrou como regularização (Ridge/L2, Lasso/L1, Elastic Net) atua como “controle de capacidade” para manter o modelo estável, simples e generalizável, evitando que a variância exploda e que coeficientes se tornem instáveis sob correlações fortes. Exploramos quando usar cada penalização (estabilização com L2, seleção automática de variáveis com L1 e equilíbrio L1+L2 em grupos correlacionados) e como calibrar hiperparâmetros com validação cruzada (k-fold) e buscas sistemáticas (GridSearchCV), sempre com padronização das variáveis numéricas.

Do ponto de vista prático, implementamos pipelines reproduzíveis com ColumnTransformer (StandardScaler nas colunas numéricas e OneHotEncoder nas categóricas), garantindo ordem fixa das transformações, prevenção de leakage e compatibilidade entre treino e produção. Em seguida, avaliamos o baseline OLS e as versões regularizadas por métricas operacionais (RMSE, MAE, R^2) para decidir quando a penalização melhora o erro fora da amostra e quando retornar ao modelo linear simples. A aula também discutiu a expansão controlada de termos polinomiais (PolynomialFeatures) e o papel da regularização para conter o risco de overfitting ao aumentar a flexibilidade do modelo.

No diagnóstico estatístico, utilizamos statsmodels para inspecionar coeficientes, erros-padrão e resíduos e aplicamos testes como Breusch–Pagan (heterocedasticidade) e Shapiro–Wilk (normalidade), além de indicadores como VIF, leverage e distância de Cook, que sinalizam multicolinearidade e pontos influentes. Essa leitura “por dentro” orienta correções (tratamento de outliers, transformações, revisão de engenharia de variáveis ou adoção de penalização) e ajuda a separar ruídos momentâneos de problemas estruturais antes de decidir por retreino ou ajustes de α /l1_ratio.

Por fim, conectamos teoria e operação: definimos prontidão, vitalidade e inicialização do pipeline como checagens de produção (carregamento de artefatos, shadow testing, bandas de RMSE/MAE/ R^2 , validação do esquema) e destacamos práticas de governança (versionamento de dados e artefatos, monitoramento de drift, rollback seguro). Também abordamos escalabilidade de treino: OLS é elegante

em bases pequenas, mas, em grandes volumes, optamos por solvers iterativos (SGD/gradiente) com regularização e boa engenharia de dados, priorizando convergência e custo.

Em síntese, você saiu desta aula com um mapa de decisões — do pré-processamento à regularização e ao MLOps — para construir regressões lineares auditáveis, eficientes e robustas em ambientes reais.



REFERÊNCIAS

BREUSCH, T. S.; PAGAN, A. R. A Simple Test for Heteroscedasticity and Random Coefficient Variation. **Econometrica**, v. 47, n. 5, p. 1287–1294, 1979. Disponível em: <https://www.econometricsociety.org/publications/econometrica/1979/09/01/simple-test-heteroscedasticity-and-random-coefficient-variation>. Acesso em: 23 jan. 2026.

EFRON, B.; HASTIE, T.; JOHNSTONE, I.; TIBSHIRANI, R. Least Angle Regression. **Annals of Statistics**, v. 32, n. 2, p. 407–499, 2004. Disponível em: <https://projecteuclid.org/journals/annals-of-statistics/volume-32/issue-2/Least-angle-regression/10.1214/009053604000000067.full>. Acesso em: 23 jan. 2026.

FRIEDMAN, J.; HASTIE, T.; TIBSHIRANI, R. Regularization Paths for Generalized Linear Models via Coordinate Descent. **Journal of Statistical Software**, v. 33, n. 1, p. 1–22, 2010. Disponível em: <https://www.jstatsoft.org/article/download/v033i01/361>. Acesso em: 23 jan. 2026.

HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. **The Elements of Statistical Learning: Data Mining, Inference, and Prediction**. 2. ed. New York: Springer, 2009. Disponível em: <https://link.springer.com/book/10.1007/978-0-387-84858-7>. Acesso em: 23 jan. 2026.

HOERL, A. E.; KENNARD, R. W. Ridge Regression: Biased Estimation for Nonorthogonal Problems. **Technometrics**, v. 12, n. 1, p. 55–67, 1970. Disponível em: <http://www.yaroslavvb.com/papers/hoerl-ridge.pdf>. Acesso em: 23 jan. 2026.

JAMES, G.; WITTEN, D.; HASTIE, T.; TIBSHIRANI, R. **An Introduction to Statistical Learning: with Applications in R**. 2. ed. New York: Springer, 2021.

KUTNER, M. H.; NACHTSHEIM, C. J.; NETER, J. **Applied Linear Regression Models**. 4. ed. New York: McGraw-Hill/Irwin, 2004. Disponível em: https://books.google.com/books/about/Applied_Linear_Regression_Models.html?id=1BqCAAACAAJ. Acesso em: 23 jan. 2026.

MONTGOMERY, D. C.; PECK, E. A.; VINING, G. G. **Introduction to Linear Regression Analysis**. 6. ed. Hoboken: Wiley, 2021. Disponível em: <https://www.wiley.com/enus/Introduction+to+Linear+Regression+Analysis%2C+6th+Edition-p-9781119578727>. Acesso em: 23 jan. 2026.

SCIKIT-LEARN. **User Guide & API Reference**. scikit-learn Documentation. Disponível em: <https://scikit-learn.org/>. Acesso em: 23 jan. 2026..

STATSMODELS. **Documentation**. statsmodels.org. Disponível em: <https://www.statsmodels.org/>. Acesso em: 23 jan. 2026..

TIBSHIRANI, R. Regression Shrinkage and Selection via the Lasso. **Journal of the Royal Statistical Society: Series B**, v. 58, n. 1, p. 267–288, 1996. Disponível em: <https://academic.oup.com/jrsssb/article/58/1/267/7027929>. Acesso em: 23 jan. 2026..

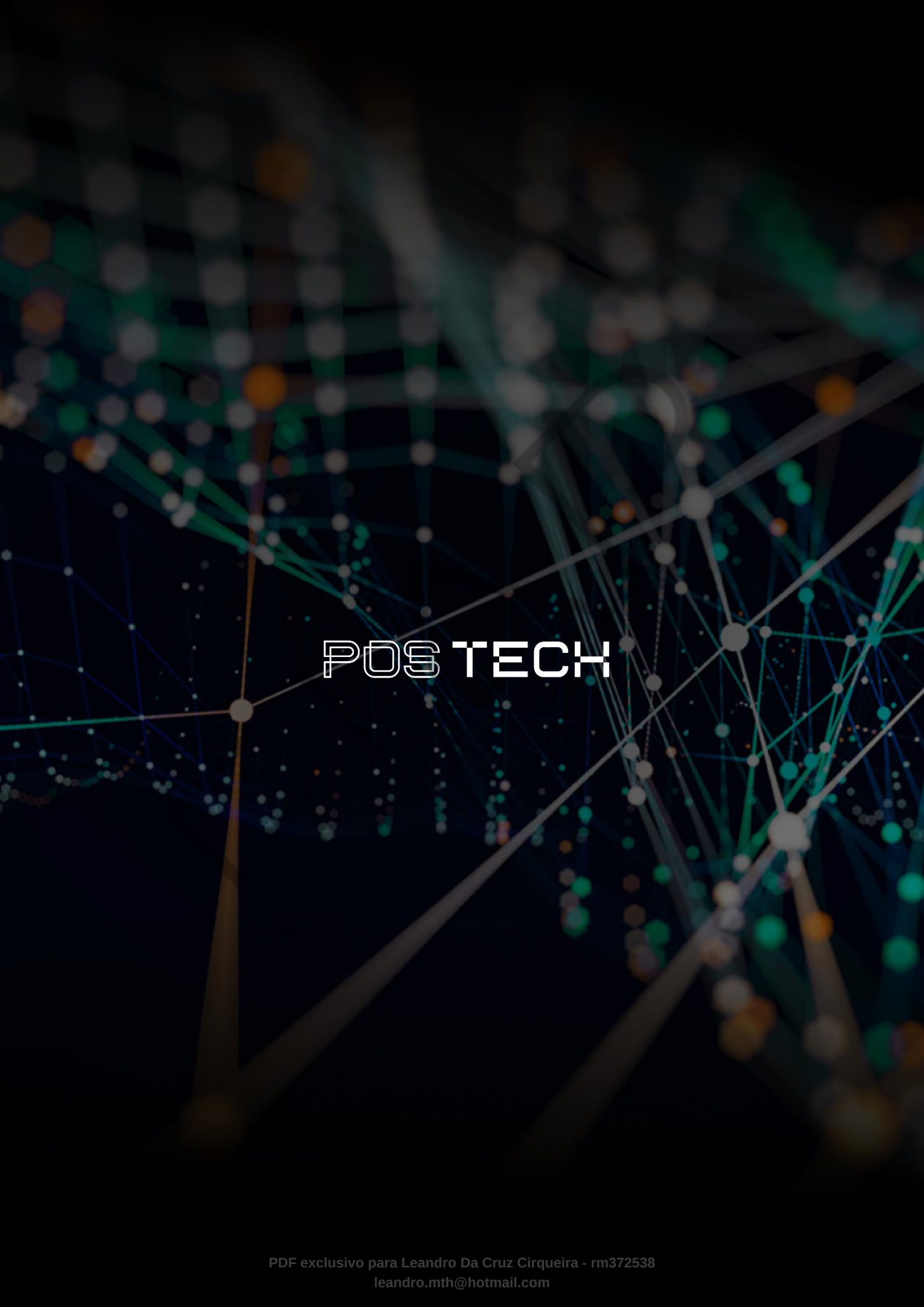
ZOU, H.; HASTIE, T. Regularization and Variable Selection via the Elastic Net. **Journal of the Royal Statistical Society: Series B**, v. 67, n. 2, p. 301–320, 2005. Disponível em: <https://hastie.su.domains/Papers/B67.2%20%282005%29%20301-320%20Zou%20&%20Hastie.pdf>. Acesso em: 23 jan. 2026.

EMAP

PALAVRAS-CHAVE

Regressão Linear. Regularização. Pipeline.

EMAP



POSTECH